DAY 27 | HANDS-ON ACTIVITIES

Agent Protocols and Advanced Use Cases: Problem Statements

Seven problems. The MCP and A2A parts run locally in Colab and VS Code; the data agent runs offline with templates or online with a model.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: MCP or A2A?

Level: Warm-up | **Concepts:** protocol choice

Decide which protocol fits: give an assistant access to a CRM; let a procurement agent ask a supplier's agent for a quote; expose a document store; delegate travel booking to a partner's agent.

Sample run / expected result

```
CRM access: MCP
Supplier quote: A2A
Document store: MCP (resources)
Travel booking: A2A
```

Hints

- Tools and data: MCP. Another agent with its own reasoning: A2A.

Problem 2: Your First MCP Server

Level: Easy | **Concepts:** FastMCP tools

Build an MCP server with get_order_status and calculate_refund tools. Start it from a client, list the tools and call both.

Sample run / expected result

```
tools: ['get_order_status', 'calculate_refund']
call get_order_status(ORD-1042) -> delivered, 4800
```

Hints

- Type hints and docstrings become the schema and description.

Problem 3: Resources and Prompts

Level: Medium | **Concepts:** MCP resources, prompts

Add a refund policy resource and a prompt template for writing apology emails. List and read them from the client.

Sample run / expected result

```
resources: ['policy://refunds']
prompts: ['apology_email']
```

Hints

- @mcp.resource("policy://refunds") and @mcp.prompt()

Problem 4: A2A-style Delegation

Level: Medium | **Concepts:** agent card, tasks

Using the notebook's simplified A2A-style agents, make the support agent discover the refund agent's card, check it has the right skill, send a task and use the artifact in its reply.

Sample run / expected result

```
card: Refund Agent, skills ['calculate_refund']
task status: completed
artifact: refund 720.0
```

Hints

- Check the skill before sending a task.

Problem 5: Handoff Payload

Level: Easy | **Concepts:** Pydantic, handoff design

Define a Handoff model and create a handoff from the bot to a human for an angry customer whose refund exceeds the bot's limit.

Sample run / expected result

```
Handoff(reason='refund above bot limit', urgency='high', steps_tried=[...], suggested_next_st
```

Hints

- Write it so the receiver needs no further context.

Problem 6: Data Analysis Agent

Level: Challenging | **Concepts:** code generation, AST checks, sandbox

Ask five questions of the sales dataset. Show the generated code, the result and a one-line explanation for each. Try two malicious snippets and confirm they are blocked.

Sample run / expected result

```
Q: highest revenue city? code: df.groupby('city')... result: Tokyo
Blocked: import os
```

Hints

- Block imports, dunder attributes, open, exec and eval.

Problem 7: Secure the MCP Server

Level: Stretch | **Concepts:** least privilege, validation, logging

Add input validation, an allowlist of order id formats, and an audit log file to the MCP server. Explain how a malicious tool result could try to manipulate the model.

Sample run / expected result

```
invalid id -> error returned
audit.log: 2026-12-01T10:02 get_order_status ORD-1042 ok
```

Hints

- Tool output is untrusted input to the model.

Ganbatte (頑張って): do your best